Modularity and Software Testing

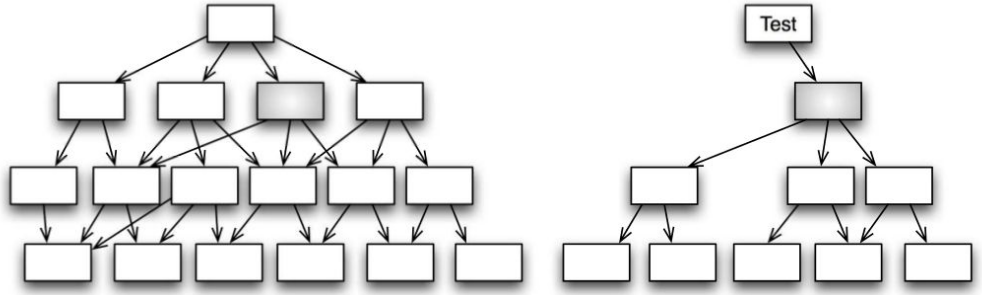
Programming and Development of Embedded Systems

Modularity & Test Double

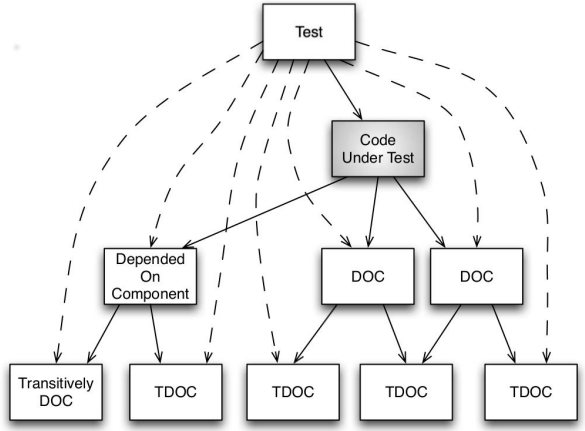
- **CUT** : Code Under Test
- A **dependency** is a function, data, module, library or device outside the CUT
  - Real code has dependencies. A module interacts with several others to do its job.
  - Dependencies make test automation difficult and expensive
- A **collaborator** is a dependency that the CUT depends upon
- A **test double** impersonates some dependencies during testing of the CUT
  - The CUT does not know it is using a test double
  - The CUT interacts with the double in the same way it interacts with the real collaborator
  - The test double is a stub taking the place of the actual production code
  - In TDD, test doubles are used to facilitate automated testing

Modularity & Test Double

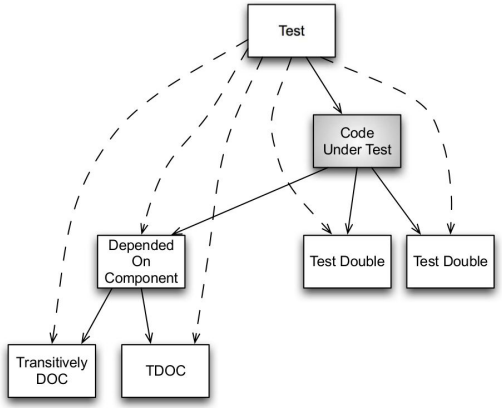
- Breaking Dependencies
  - Using of interfaces
  - Using of hierarchical design
  - Encapsulation and hiding details
  - Less reliance on unprotected global data



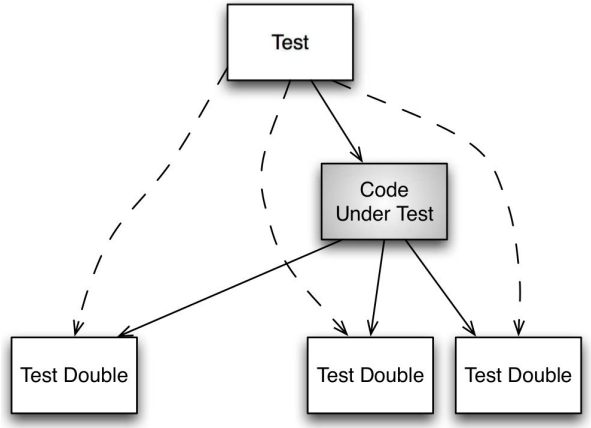
Dependencies of the gray module



Test Dependency Mess



Using test doubles and real collaborators



Manage test dependencies with test doubles

Akademin

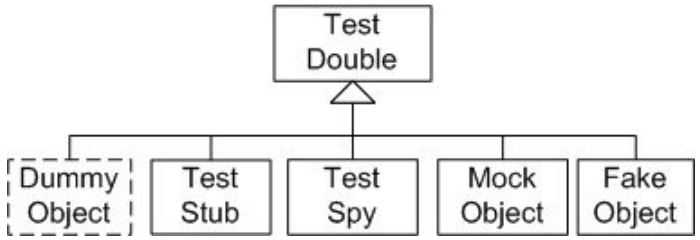
Modularity & Test Double

- The rule of thumb
  - Use the real code when you can and use a test double when you must
- When to Use a Test Double
  - Hardware independence
    - Testing independently from the hardware
    - Providing a wide variety of inputs into the core of the system that may be difficult to do it in the lab
  - Inject difficult to produce input(s)
    - Some computed or hardware-generated event scenarios may be difficult to produce
  - Speed up a slow collaborator
    - A slow collaborator, such as a database or a network service

Modularity & Test Double

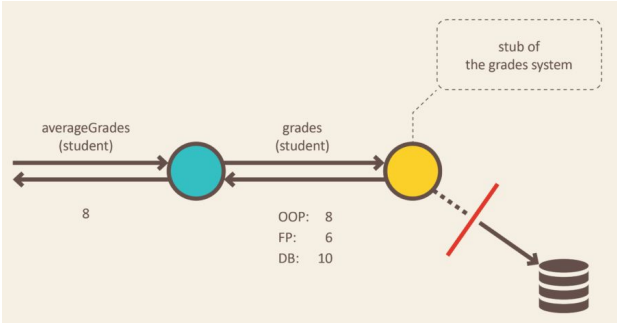
- When to Use a Test Double ...
  - Dependency on something under development
  - Dependency on something volatile
    - Like an alarm clock.
    - You have an event that is supposed to get triggered at 03:42
  - Dependency on something that is difficult to configure
    - A database is an example of a DOC that you could test with but is difficult to set up

- Test Double Variations
  - There are different types of doubles like
    - Test Stub, Test Spy, Fake Object, Mock Object and etc.



Modularity & Test Double Variations

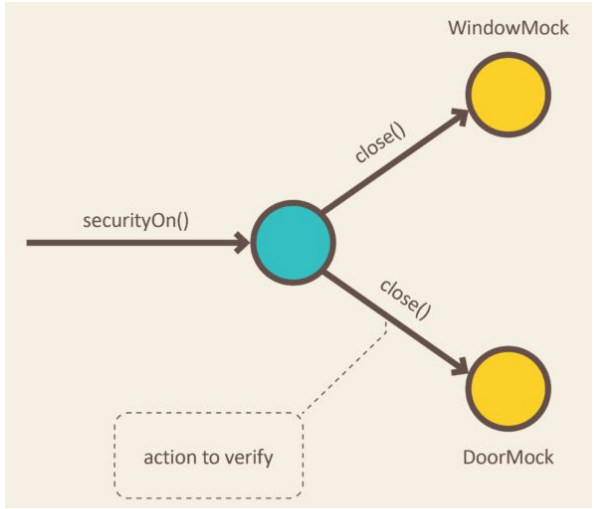
- Test dummy
  - It is provided to satisfy the compiler, linker or runtime dependency
- Test stub
  - It returns a value, as directed by the current test case
  - It holds predefined data and uses it to answer calls during tests
  - It is used when we cannot or don't want to involve objects that would answer with real data

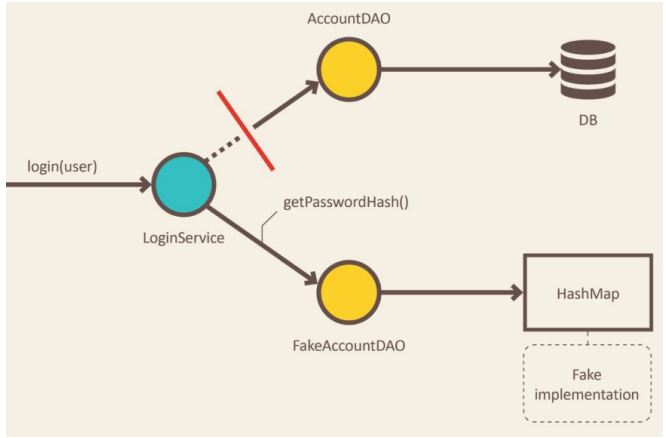


- Test spy
  - It captures the arguments passed from the CUT so the test can verify that the correct arguments have been passed to the CUT
  - It can also feed return values to the CUT just like a test stub

Modularity & Test Double Variations

- Mock Object
  - It verifies function calls, call orders, and the arguments passed from the CUT to the DOC
  - It usually deals with a situation where multiple calls are made to it, and each call and response are potentially different
- Fake Object
  - It provides a partial and simplified version of the real dependency
  - It is used when the CUT depends on a component which makes testing difficult or slow

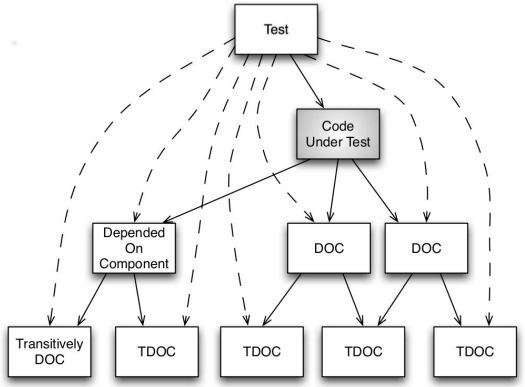


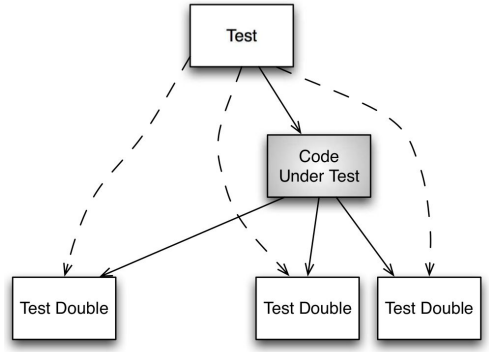


Akademin

Faking Techniques

- The primitive mechanisms to fake dependencies in C/C++
  - Preprocessor-time substitution
  - Compile-time substitution
  - Link-time substitution
  - Run-time substitution using function pointers
- Link-time substitution
  - Use it when you need to
    - Replace the DOC for the whole unit test,
    - Substitute a module where you do not control the interface
    - Eliminating dependencies on third-party libraries, hardware-dependent modules, or the OS
  - Instead of linking the real module, we need to make a test double for the DOC and link it





Faking Techniques

- Function pointer or pure virtual function substitution
  - Use it when you want to replace a depencey for only some of the test cases
  - You can use it everywhere that you control the interface
  - It uses some RAM, and compromises function declaration readability
  - It allows great control over where and when functions get overridden
- Combination of link-time and function pointer substitutions
  - We can combine link-time and function pointer substitution to work together
  - A link-time stub can be created that contains function pointer
  - A test case can override the function pointer and provide exactly the stub function
  - You want the flexibility of the function pointers but not to change the interface of the depencey
- Preprocessor/inline-function substitution
  - Use it when linker and function pointer substitutions can’t do the job
  - You can break a chain of unwanted includes with preprocessors
  - You can also use it selectively or temporarily to override names

```
typedef struct {
  void *(*malloc)(size_t);
  void (*free)(void *);
} memory_t;

struct IMemory {
  virtual void *malloc(size_t size) = 0;
  virtual void free(void *ptr) = 0;
  virtual ~IMemory() = default;
};
```

Code Coverage

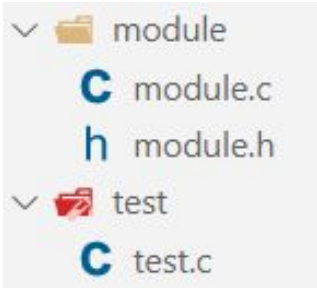
- What is Code Coverage?
  - Is the percentage of the code which is covered by automated tests
  - Determines which parts of the code is executed through a test run, and which parts is not
- Why Measure Code Coverage?
  - To know how well our tests actually test the production code
  - To know whether we have enough testing in place
  - To maintain the test quality over the lifecycle of the project
- Types of Coverage measured
  - **Statement coverage**: measures whether each statement is executed
  - **Branch coverage**: measures which branches (e.g. if-else) in flow control structures are followed
  - **Function coverage**: measures whether each function in the program been called

Code Coverage

- **Code instrumentation** refers to insert code in programs to monitor
  - Code tracing, debugging, analyzing, performance measuring, data logging and etc.
- Mainly there are three code Instrumentation approaches
  - **Source-code** instrumentation: Add the instrumentation code to the source code and compile the it with the compilation tool chain to produce an instrumented program.
    - Is done by developers and processed in the preprocessor step.
  - **Compile-time** instrumentation: Is done during compilation of source files, and the instrumentation is performed by the compiler.
    - E.g. debugging(-g), profiling(-pg), code coverage(--coverage), and etc.
  - **Runtime** instrumentation: The instrumentation code and data are inserted into the executable binary files during execution. It is done during runtime.
    - E.g. Dynamic binary instrumentation tools like Valgrind

Code Coverage

- We need to use **gcov** to instrument coverage code in files
  - **gcov** is the code coverage library for **gcc/g++**.
  - Compile source files with **--coverage**
  - Link the test program with **--coverage** or link **-lgcov**
  - Run the test to produce .gcda and .gcno files
- To generate code coverage reports:
  - You can use **gcovr**(a python package). To install gcovr: ***sudo apt install gcovr***
  - Run **gcovr -r <*root-directory*> --html-details -o <*output-directory*>/index.html**
    - Specify root-directory and the output-directory of your project. Look at the gcovr options



- Example
  - mkdir build && gcc -c --coverage module/module.c -o build/module.o
  - gcc -c test/test.c -I./module -o build/test.o; gcc --coverage build/module.o build/test.o -lunity -o build/test
  - build/test && gcovr -r . --html-details -o build/index.html

Dual Targeting Strategy

- Dual-targeting means that codes are designed to run on two platforms:
  - The development system
  - The embedded target
- Concurrent hardware and software development is a reality for many projects
- Dual targeting strategy is used because:
  - The target hardware is expensive
  - The target hardware is not ready until late in the project
  - When the target hardware is finally available, it may have bugs of its own
  - Long target build or upload times waste valuable time during the development cycle
  - Compilers for the target hardware are considerably more expensive than native compilers
  - And etc.

Dual Targeting Strategy

- Benefits of Dual Targeting
  - It allows us to overcome the hardware restrictions
  - Dual targeting, like TDD, produces more modular and hardware independent designs
- Risks of Dual Target Testing
  - Differences between the development and target environments
  - Compilers may support different language features
  - The runtime libraries may be different
  - Primitive data types might have different sizes
  - The target compiler may have bugs
  - Byte ordering and data structure alignments may be different

Software Testing

- Some useful links
  - Effective Tests: Test Doubles
  - Test Doubles — Fakes, Mocks and Stubs
  - Test Double
  - gcovr documentation
  - About Code Coverage
  - Instrumentation (computer programming)
  - The Ultimate List of Code Coverage Tools
  - Code Coverage Using GNU Gcov and Lcov
  - Using gcov and lcov
  - Code instrumentation